Project Testing Guide

Embedded Security – SSH Tunneling for Modbus/TCP

This document outlines the official test cases required to verify the functionality, performance, and security of the Modbus/TCP SSH Tunneling project.

Prerequisites

1. Open your terminal and navigate to the project directory:

```
cd "/Users/vedha/Documents/ESD - Modbus:TCP"
```

2. Ensure all dependencies are installed:

```
make install
```

Test Case 1: Functional Testing (Connectivity)

This test proves that the Modbus Server and Client can successfully communicate under all three conditions.

1A: Plaintext (Direct Mode)

- **Terminal 1:** Run make server (Leave running)
- **Terminal 2:** Run make client
- *Verification:* The client should output successful read and write operations.

1B: Native Tunnel (OS Level)

- **Terminal 1:** Run make server (Leave running)
- **Terminal 3:** Run make tunnel-native (Enter your Mac password when prompted, leave running)
- **Terminal 2:** Run make client-tunnel
- *Verification:* The client successfully connects to port 2222 and completes its operations.

1C: Python Tunnel (Application Level)

- **Terminal 1:** Run make server (Leave running)
- **Terminal 3:** Press Ctrl+C to stop the previous tunnel, then run make tunnel-python (Enter credentials, leave running)
- **Terminal 2:** Run make client-tunnel
- *Verification:* The client successfully connects to port 2222 and completes its operations.

Test Case 2: Performance Testing (Overhead Analysis)

This test proves the latency and CPU overhead difference between the three routing methods.

2A: Automated Benchmark

- **Terminal 1:** Run make server (Leave running)
- **Terminal 2:** Run make benchmark
- *Execution Steps:*

1. The script will automatically benchmark the Plaintext connection.
2. It will pause. Go to Terminal 3, run make tunnel-native, and press Enter in Terminal 2.
3. It will pause again. Go to Terminal 3, press Ctrl+C, run make tunnel-python, and press Enter in Terminal 2.

- *Verification:* The console prints latency and CPU metrics, and a new 3-column bar chart is successfully generated at docs/benchmark_results.png.

Test Case 3: Security Verification (Packet Inspection)

This test proves to an external observer that the tunnel successfully hides the sensitive Modbus data payload.

3A: Proving the Vulnerability (Plaintext)

1. Open the **Wireshark** application.
2. Capture traffic on the **Loopback (lo0)** interface.
3. Apply the display filter: tcp.port == 5020
4. Run Test Case 1A (make client).
5. *Verification:* Wireshark displays green packets labeled Modbus/TCP. The function codes and register values are clearly readable in plaintext.

3B: Proving the Solution (Encrypted Tunnel)

1. In **Wireshark**, change the display filter to: tcp.port == 22
2. Run Test Case 1B or 1C (Start a tunnel and run make client-tunnel).
3. *Verification:* Wireshark displays packets labeled SSHv2. The Modbus payload is completely hidden within the encrypted data stream, proving the connection is secure.